Large Language Models in Data Science

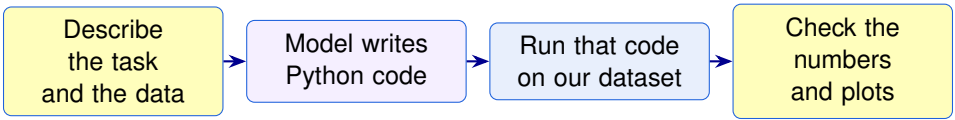
Week 3: From a Task to a Verified Analysis

Sebastian Mueller
Aix-Marseille Université
2026–2027

From Using a Model to Checking Its Work

Week 2: load a trained model, interpret its outputs and measure its performance.

Today: ask a hosted model to write Python for an analysis, run that code on our data, and check the result.



The model returns code as text. Python performs the calculation on the actual data; we check that it answers the question correctly.

Start with a Question We Can Check

The assignment

Customers are leaving. Does observed churn differ by contract type?

Churn means that a customer leaves. In our data, `churn=1` means the customer left; `churn=0` means they did not.

For each contract type, we want:

- the number of customers and the number who left;
- the fraction who left: the number who left divided by the total number of customers with that contract type.

A Small Dataset We Can Check by Hand

`csv_text` is a Python string containing comma-separated values (CSV): six customers, one row each.

```
csv_text = """contract_type,churn
month-to-month,1
month-to-month,0
month-to-month,1
one-year,0
one-year,1
two-year,0
"""
```

Context we must supply

- What a row represents.
- What the columns and values mean.
- Which rows belong in each calculation.

Check now: among the three month-to-month customers, how many left?

Later, Python will analyse the lab's 600-row synthetic dataset. We use these six rows first to make errors visible.

Turn the Question into an Explicit Request

`task_text` is a Python string describing the user's task. This variable name is our choice; it is not an API field.

```
task_text = """Compare churn by contract type.
Each row is one customer; churn: 0 = stayed, 1 = left.
For each type, report n_customers (group size)
and n_churn (customers who left).
churn_rate = n_churn / n_customers, from 0 to 1.
Return a table. Report observed differences only."""
```

This specifies the **task**, relevant **context**, required **output** and a limit on the **conclusion**.

Start with what the task needs. Add instructions or examples to address failures you actually observe.

Why Separate System Instructions from the User Message?

Imagine an analysis application used for different datasets and questions. We separate the application's guidance from the user's particular task.

System instruction

Set by the application

Use supplied evidence. State missing information. Distinguish observation from causation.

User message

The task and its data

Compare churn by contract type using the six CSV rows and the definitions we just supplied.

The application can reuse the guidance while the user's task and data change.

In our example, both parts are sent together on every call. The system instruction is optional; one user message is enough for a simple request.

Connect Python to a Hosted Model

An **API** is an interface through which our Python program sends a request to a service and receives a response.

```
import os
from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()  # Load local settings from .env.
client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
model_name = "gemini-3.5-flash-lite"  # Common model for this course.
```

- `client` is the object we use to contact the service.
- `model_name` fixes one common model for the course.
- The model runs at the provider; we do not download its weights here.

Build the User Message

```
task_text = """Compare churn by contract type.
Each row is one customer; churn: 0 = stayed, 1 = left.
For each type, report n_customers (group size)
and n_churn (customers who left).
churn_rate = n_churn / n_customers, from 0 to 1.
Return a table. Report observed differences only."""

csv_text = """contract_type,churn
month-to-month,1
month-to-month,0
month-to-month,1
one-year,0
one-year,1
two-year,0
"""
prompt = task_text + "\n\nData (CSV):\n" + csv_text
```

Send the Message and Read the Table Reply

```
config = types.GenerateContentConfig(
    system_instruction=(
        "Use supplied evidence. State missing information. "
        "Distinguish observation from causation."))
response = client.models.generate_content(
    model=model_name, contents=prompt, config=config,
)
print(response.text)
```

This call sends the system instruction through `config`, and the user message (task plus six CSV rows) through `contents=prompt`.

`response.text` contains the model's table reply as a Python string. `print(response.text)` displays that reply.

What Does the Table Reply Look Like?

```
print(type(response.text))
print(response.text)
```

A possible output, illustrated with our six rows:

```
<class 'str'>
| contract_type   | n_customers | n_churn | churn_rate |
|-----------------|-------------|---------|------------|
| month-to-month | 3           | 2       | 0.6667     |
| one-year       | 2           | 1       | 0.5        |
| two-year       | 1           | 0       | 0.0        |
```

`str` means a Python string. The table is written using characters inside that string; the bars separate its columns.

We can read the displayed table, but Python needs to extract its values before using them in calculations. Next, we request a format designed for that step.

Choose an Output Your Program Can Read

For the next call, ask for the same results as **JSON text**. JSON is a text format for objects with named fields, lists and values.

```
[
  {"contract_type": "month-to-month",
   "n_customers": 3, "n_churn": 2,
   "churn_rate": 0.6667}
]
```

Illustrative output for **one group**; the complete answer needs all three groups.

- Outer brackets hold a list; braces hold an object.
- `contract_type` is a string; counts and rates are numbers.
- An example clarifies the convention: `0.6667` is a fraction, not `66.67` percent.

Parsing Is Only the First Check

```
import json
json_prompt = prompt.replace("Return a table.", "Return a JSON list.")
json_response = client.models.generate_content(
    model=model_name, contents=json_prompt, config=config,
)
rows = json.loads(json_response.text)
```

We expect a Python list of dictionaries, one per contract type.

Check	**Question**	**How we check**
Syntax	Can Python read the JSON?	`json.loads()`
Structure	Correct fields and types?	Validate against a schema.
Content	Correct counts and rates?	Recompute from the data.

A **schema** defines required fields, their types and limits.

Structured Output: First Define the Required JSON

Structured output is an API feature. We supply a **JSON Schema**: a specification of the response's structure, fields and types.

```
row_schema = {
    "type": "object",
    "properties": {
        "contract_type": {"type": "string"},
        "n_customers": {"type": "integer", "minimum": 1},
        "n_churn": {"type": "integer", "minimum": 0},
        "churn_rate": {"type": "number", "minimum": 0, "maximum": 1},
    },
    "required": ["contract_type", "n_customers", "n_churn", "churn_rate"],
}
result_schema = {"type": "array", "items": row_schema}
```

`properties` defines the fields; `required` lists mandatory fields. `array` means a list, whose items must follow `row_schema`.

Enable Structured Output in the API Call

```
json_config = types.GenerateContentConfig(
    system_instruction=config.system_instruction,
    response_mime_type="application/json",
    response_json_schema=result_schema,
)
json_response = client.models.generate_content(
    model=model_name, contents=json_prompt, config=json_config,
)
rows = json.loads(json_response.text)
```

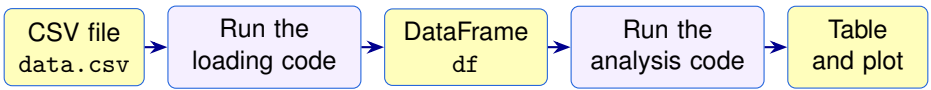
- `response_mime_type` selects JSON as the response format.
- `response_json_schema` supplies the required structure and types.

The reply is still JSON text. `json.loads()` converts it to Python objects.

`0.99` would pass the range check (0 to 1), however, the correct rate is $2/3 \approx 0.667$.

From a CSV to an Analysis Workflow

Until now, the model returned a table and then a json file. Now we ask it to write the Python code that loads the data and calculates the statistics.



Step 1: Ask for Code That Loads the CSV

```
load_prompt = """Write Python defining load_data(path).
Read the CSV file at path with pandas and return a DataFrame.
Keep every row and both columns, with the original names.
Return only Python code, with imports and no Markdown.
The file contains:
""" + csv_text

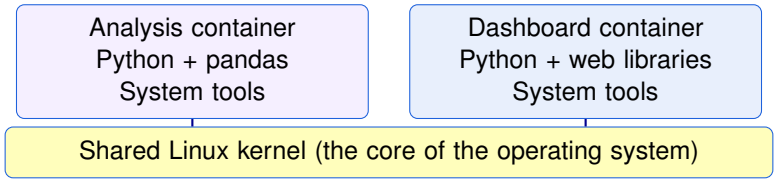
load_response = client.models.generate_content(
    model=model_name, contents=load_prompt, config=config,
)
loader_code = load_response.text
print(loader_code)
```

`path` will be the file's location when the function runs. The model receives the CSV text in the message; it does not open our local file.

`loader_code` contains Python code as a string. We inspect and execute it next.

Docker: Package and Run Applications Independently

Docker packages an application and the software it needs into an **image**: for example, Python, libraries and system tools. A **container** runs that image with its own files and processes.



- **Modularity**: run and update each application separately.
- **Reproducibility**: reuse the same image to keep the software environment consistent across runs and computers.

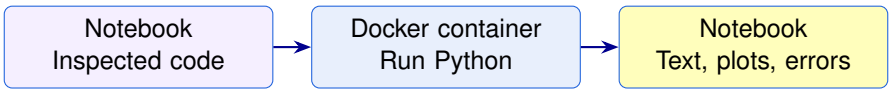
A `venv` separates Python packages. Docker also packages system tools and provides separate files and processes for each application.

Generated Code: Restrict What It Can Access

The model returns code we have not yet verified. Running it in our notebook would give it the notebook's access to files and the network.

We use a **sandbox**: an execution environment with restricted access and resources. Our helper configures a Docker container for this purpose:

- expose the selected CSV read-only; keep other personal files and API keys outside;
- disable network access during execution;
- limit execution time and memory.



Run the Loading Code and Inspect the DataFrame

Inspect the generated function, then execute it in the Docker container. For example:

Code executed in the container

```
import pandas as pd

def load_data(path):
    return pd.read_csv(path)

df = load_data("/input/data.csv")
print(df)
```

Printed DataFrame

	contract_type	churn
0	month-to-month	1
1	month-to-month	0
2	month-to-month	1
3	one-year	0
4	one-year	1
5	two-year	0

Check: six rows, two correctly named columns, and unchanged values.

The numbers 0 to 5 are row indices, not an extra column in the CSV.

Step 2: Ask for Code That Analyses the DataFrame

```
analysis_prompt = """Write Python defining analyze(df).
Each row is one customer; churn: 0 = stayed, 1 = left.
For each contract_type, print n_customers (group size),
n_churn (number who left), and churn_rate = n_churn / n_customers.
Make a labelled bar chart of those rates, from 0 to 1.
Use pandas and matplotlib, with any necessary imports.
Do not modify df or call plt.show() or plt.close().
Return only Python code, no Markdown.
The CSV loaded into df is:
""" + csv_text
analysis_response = client.models.generate_content(
    model=model_name, contents=analysis_prompt, config=config,
)
analysis_code = analysis_response.text
print(analysis_code)
```

Inspect this code, then call `analyze(df)` with the DataFrame we just loaded.

Code for the Six-Row Example

This function calculates the group counts and rates, prints the table, and creates a bar chart:

```
import matplotlib.pyplot as plt
plt.rcParams.update({"font.size": 14})

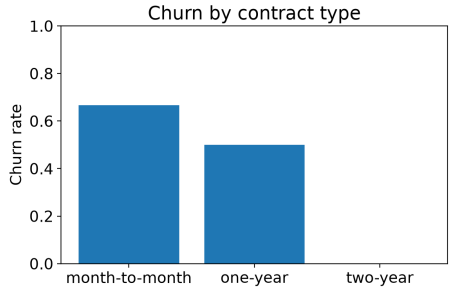
def analyze(df):
    summary = df.groupby("contract_type", sort=False)["churn"].agg(
        n_customers="size", n_churn="sum").reset_index()
    summary["churn_rate"] = summary["n_churn"] / summary["n_customers"]
    print(summary.to_string(index=False, float_format="{:.3f}".format))
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(summary["contract_type"], summary["churn_rate"])
    ax.set(ylabel="Churn rate", ylim=(0, 1))
    ax.set_title("Churn by contract type")
    fig.tight_layout()
```

Reviewed example, not a captured API reply. The following results come from executing this code.

Run the Code: Table, Plot and a Check

Printed output

contract_type	n_customers	n_churn	churn_rate
month-to-month	3	2	0.667
one-year	2	1	0.500
two-year	1	0	0.000



Check directly against the six CSV rows:

- Counts: $3 + 2 + 1 = 6$.
- Rates: $2/3$, $1/2$ and $0/1$.

The counts and rates are correct for these six rows. If a check failed, we would send the previous code and the failed check back to the model, then execute and check the revision.

Ask for a More Readable Plot

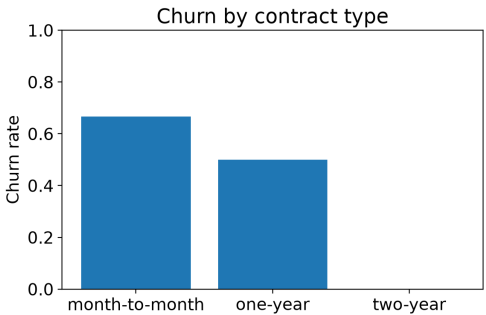
The numbers are correct. Label the bars so the zero rate is explicit:

```
feedback = """Keep the table calculations unchanged.
Use horizontal bars in teal.
Show percentages on the axis and label every bar, including zero.
Keep the scale from 0% to 100%.
Return the complete revised analyze(df) function, with imports."""
revision_prompt = (
    analysis_prompt + "\n\nPrevious code:\n" + analysis_code
    + "\n\nRequested changes:\n" + feedback
)
revised_response = client.models.generate_content(
    model=model_name, contents=revision_prompt, config=config,
)
revised_code = revised_response.text
print(revised_code)
```

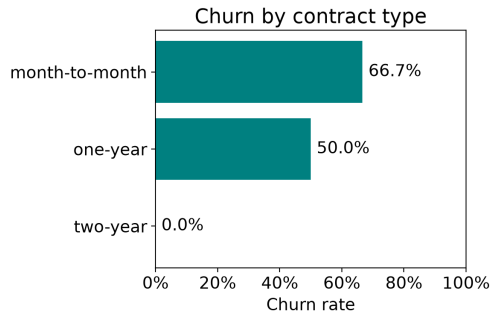
Run the revised code and check the counts and rates again.

Revised Plot: Same Numbers, Clearer Labels

First version



Revised version

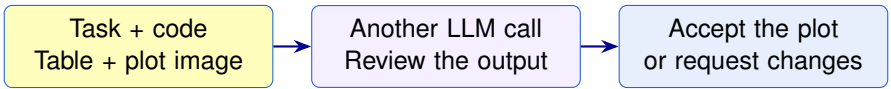


Checked after revision: the printed counts and rates are unchanged. The bars now run horizontally, and percentages label every group, including 0%.

Ask an LLM to Review the Plot

We just inspected the plot ourselves and requested a change. We can automate this review with another LLM call.

Send the task, previous code, printed table and actual PNG image to a model that accepts images:



The review request

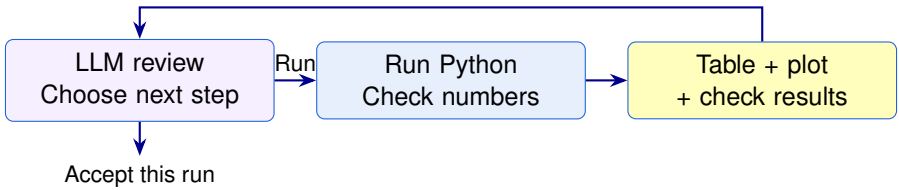
Compare the plot with the table. Are the axes and units clear? Is every group labelled, including zero values? Return specific changes if needed; otherwise accept the plot.

Python checks the numbers independently. The LLM reviews readability and consistency with the table; its feedback can trigger a new code-generation call.

From Our Review to an Agent Loop

In an **agent loop**, the model uses returned results to choose its next action: revise the code, run again or accept the result.

We implement this loop in Week 5: the program runs the code and returns results.



After the six-row check passes: run the same code on the full dataset and check that output too.

Finish when the full analysis passes the numerical checks and meets the presentation requirements. Limit revisions, for example to three attempts; unresolved issues go to human review.

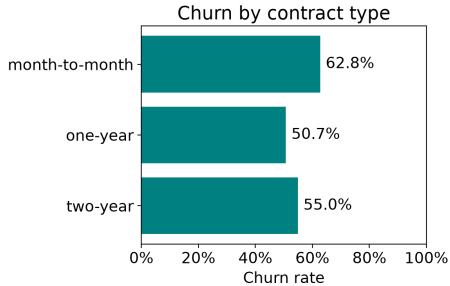
Finally, Run the Checked Code on All 600 Rows

`df_full` contains the full synthetic customer dataset. Run the same `analyze(df_full)` function and check its output:

Printed output on the full dataset

contract_type	n_customers	n_churn	churn_rate
month-to-month	333	209	0.628
one-year	136	69	0.507
two-year	131	72	0.550

Check: $333 + 136 + 131 = 600$ customers. Each rate matches an independent count from the full CSV.



Send the Data, or Generate Code to Process Them?

Same question: customer counts, churn counts and churn rates for each contract type.

	Full CSV in the prompt	**Generate Python, run locally**
Send to the LLM	Task + all 600 rows	Task + column definitions + 6 example rows
Receive	A table of results	Code that calculates the table
Next CSV	Send the new data again	Reuse the checked code if the task and columns are unchanged

Lab comparison: use the same model, check both results, and measure input tokens, output tokens and API cost. Code generation also produces output tokens; count review and revision calls too.

Further Reading: Instructions, Outputs and Feedback

- Google: prompt design strategies
  Clear instructions, relevant context and examples.
- Google: structured outputs
  Schemas for program-readable results; content still needs checking.
- Anthropic: effective context engineering for AI agents
  Choose what each call needs, including tool results and history.
- Anthropic: building effective agents
  Fixed workflows and agents that choose actions using feedback.

API examples follow the Google Gen AI Python SDK. Sources checked 19 October 2026.

Before the Lab: Get Your Gemini API Key

1. Open aistudio.google.com/api-keys and sign in with a Google account.
2. Copy an existing key, or click **Create API key** and select or create a project. Accept the terms if prompted.
3. Save the key in `.env`, next to the repository's `README.md`.

```
GEMINI_API_KEY=paste_your_key_here
```

Model used in the lab: `gemini-3.5-flash-lite`.

Keep the key private. The notebook reads `.env`; do not put your key in a notebook cell or commit it to Git.

Set `RUN_LIVE = True` in the notebook to enable API requests.

Google's API-key setup guide

Checked 20 October 2026.